

Planificateur de révisions

Génération automatique de plannings de révision
par répétition espacée

Rapport technique

Mohamed Aziz Baoueb

Stage réalisé chez **Anypli** — Tunisie
Juillet — Août 2025

Refonte complète : octobre 2026

Résumé

Ce document décrit la conception et la réalisation d'une application web destinée aux étudiants : à partir d'une liste d'examens, de leurs coefficients et de la difficulté des matières concernées, elle produit automatiquement un planning de révision réparti dans le temps selon le principe de la *répétition espacée*.

Le projet trouve son origine dans un stage d'un mois effectué chez **Anypli**, en Tunisie, pendant l'été 2025. Je n'avais alors aucune expérience du développement web : TypeScript, React, Redux et Strapi ont tous été découverts sur place, à partir de zéro, en quatre semaines. Le présent rapport retrace cet apprentissage, puis la refonte complète menée ultérieurement, qui a transformé un prototype fonctionnel en une application structurée, testée et documentée.

Le document assume volontairement un regard critique sur la première version : en exposer les faiblesses est la manière la plus honnête de mesurer le chemin parcouru.

Table des matières

1	Contexte	3
1.1	Le stage chez Anypli	3
1.2	Le besoin	3
1.3	Objectifs	3
2	Les technologies découvertes	5
2.1	TypeScript	5
2.2	React	5
2.3	Redux Toolkit	5
2.4	Strapi	5
2.5	Les autres outils	6
3	La version réalisée pendant le stage	7
3.1	Ce qui a été livré	7
3.2	Les limites, avec le recul	7
3.2.1	Un backend non versionné	7
3.2.2	Un modèle de données incertain	7
3.2.3	Un filtrage de sécurité illusoire	8
3.2.4	Une erreur de précedence	8
3.2.5	Un algorithme de planification rudimentaire	8
3.2.6	Autres points	8
4	La refonte	9
4.1	Principes retenus	9
4.2	Architecture	9
4.3	Modèle de données	9
5	Le moteur de planification	11
5.1	Volume de révision	11
5.2	Répétition espacée	11
5.2.1	Adaptation à la fenêtre disponible	12
5.3	Placement des séances	12
5.4	Garanties	12
5.5	Diagnostics	12
5.6	Validation des entrées	12
6	Le backend	13
6.1	Permissions appliquées par le code	13
6.2	Cloisonnement des données	13
6.3	Données de démonstration	13

7	L'interface	14
7.1	Prévisualisation avant enregistrement	14
7.2	Déclaration des disponibilités	14
7.3	Notifications	15
8	Qualité	16
8.1	Tests	16
8.2	Démonstration reproductible	16
9	Difficultés rencontrées	17
9.1	Pendant le stage	17
9.2	Pendant la refonte	17
10	Bilan et perspectives	18
10.1	Ce que le stage a apporté	18
10.2	Ce que la refonte a apporté	18
10.3	Perspectives	18

Chapitre 1

Contexte

1.1 Le stage chez Anypli

Ce projet a été réalisé dans le cadre d'un stage d'un mois au sein de l'entreprise **Anypli**, en Tunisie, durant l'été 2025. L'objectif du stage était double : découvrir le développement web moderne dans un cadre professionnel, et livrer une application complète de bout en bout.

Le point de départ mérite d'être souligné : **je n'avais aucune connaissance préalable** des technologies employées. TypeScript, React, la gestion d'état par Redux, l'architecture d'une API REST, le fonctionnement d'un CMS découplé comme Strapi — tout a été appris pendant ces quatre semaines, en même temps que l'application se construisait. L'historique Git du dépôt en conserve la trace : les premiers essais datent de la mi-juillet 2025, la version livrée du 15 août 2025.

Apprendre une pile technique entière et produire un livrable dans le même mois impose des compromis. La première version fonctionnait et répondait au besoin, mais elle portait les marques de cet apprentissage accéléré. C'est ce constat, assumé, qui a motivé la refonte décrite au chapitre 4.

1.2 Le besoin

Un étudiant connaît ses échéances : il sait qu'il a un partiel de mathématiques dans trois semaines, un oral d'anglais dans dix jours, un projet à rendre dans un mois. Ce qu'il sait mal faire, c'est *répartir son travail* entre ces échéances.

Les difficultés sont connues :

- la tendance à tout réviser la veille, ce que la recherche sur la mémoire identifie comme la stratégie la moins efficace ;
- la difficulté à pondérer l'effort selon l'importance réelle de chaque épreuve ;
- l'oubli pur et simple des matières dont l'échéance est lointaine ;
- la planification de séances à des moments où l'on n'est, en pratique, pas disponible.

L'application répond à ce besoin : l'étudiant saisit ses examens et déclare ses créneaux de disponibilité ; le système produit un planning concret, affiché dans un calendrier.

1.3 Objectifs

Objectif pédagogique

Maîtriser une pile complète : un langage typé (TypeScript), une bibliothèque d'interface (React), une gestion d'état (Redux Toolkit) et un backend (Strapi), ainsi que le dialogue entre les deux côtés via une API REST.

Objectif fonctionnel

Livrer une application utilisable : authentification, gestion des examens, génération et visualisation d'un planning.

Objectif de qualité

Atteint lors de la refonte uniquement : rendre le projet reproductible, testé et documenté.

Chapitre 2

Les technologies découvertes

Ce chapitre décrit les technologies employées et, pour chacune, ce qu'elle apporte concrètement au projet.

2.1 TypeScript

TypeScript ajoute un système de types statiques à JavaScript. Son intérêt est de déplacer la détection des erreurs de l'exécution vers la compilation.

La configuration retenue après refonte active les options les plus strictes :

Listing 2.1 – Options de compilation, `tsconfig.json`

```
{
  "strict": true,
  "noUncheckedIndexedAccess": true,
  "exactOptionalPropertyTypes": true,
  "noImplicitOverride": true,
  "noUnusedLocals": true,
  "verbatimModuleSyntax": true
}
```

`noUncheckedIndexedAccess` mérite une mention : il force à traiter `tableau[i]` comme potentiellement `undefined`. C'est contraignant à l'écriture, mais cela élimine une classe entière d'erreurs à l'exécution.

2.2 React

React construit l'interface à partir de composants : des fonctions qui reçoivent des données et retournent une description de l'affichage. Le projet utilise React 18 avec les *hooks* (`useState`, `useEffect`, `useMemo`).

2.3 Redux Toolkit

Redux centralise l'état de l'application dans un *store* unique, modifié par des actions. Redux Toolkit en réduit considérablement la verbosité. Le projet l'emploie pour l'authentification, la liste des examens, les données de référence et les notifications.

2.4 Strapi

Strapi est un CMS *headless* : il fournit une base de données, une API REST, une authentification et une interface d'administration, à partir de la seule description des types de contenu.

C'est ce qui permet, en un mois, de disposer d'un backend complet sans l'écrire entièrement à la main.

La version 5 introduit le `documentId`, un identifiant textuel stable qui remplace l'identifiant numérique dans l'API publique.

2.5 Les autres outils

Outil	Rôle	Version
Vite	serveur de développement et empaquetage	5
FullCalendar	affichage du calendrier	6
i18next	internationalisation français / anglais	23
Vitest	exécution des tests	2
Playwright	pilotage du navigateur pour la démonstration	1.49
SQLite	base de données de développement	—
PostgreSQL	base de données de production (Docker)	16

TABLE 2.1 – Outils employés

Chapitre 3

La version réalisée pendant le stage

3.1 Ce qui a été livré

À l'issue du mois, l'application permettait de créer un compte, de se connecter, de saisir des examens avec une date et un coefficient, de générer un planning de révision et de le visualiser dans un calendrier. L'interface était traduite en français et en anglais. Le backend Strapi gérait les relations entre l'utilisateur, l'élève, les examens et les sessions.

Pour un premier contact avec ces technologies, le résultat était fonctionnel.

3.2 Les limites, avec le recul

Reprendre ce code un an plus tard permet d'en identifier les faiblesses. Les exposer n'est pas un désaveu : c'est la mesure de ce qui a été appris depuis.

3.2.1 Un backend non versionné

La configuration Strapi — types de contenu, relations, permissions — avait été réalisée à la main dans l'interface d'administration. Elle ne figurait donc nulle part dans le dépôt. Conséquence directe : **personne ne pouvait lancer le projet**. Un clone donnait un écran vide et des erreurs 403.

3.2.2 Un modèle de données incertain

Faute de maîtriser le schéma qu'il interrogeait, le code tentait plusieurs noms de relation jusqu'à ce que l'API cesse de répondre une erreur :

Listing 3.1 – Extrait de la version 1 — le code devine son propre modèle

```
const REL_KEYS = ['eleves', 'eleve', 'students'] // essaye plusieurs noms possibles
for (const key of REL_KEYS) {
  try {
    const { data } = await api.get('/api/exams', {
      params: { filters: { [key]: { user: { id: { $eq: userId } } } } }
    })
    fromServer = data?.data ?? []
    break
  } catch (e) { /* mauvais nom : on tente le suivant */ }
}
```

3.2.3 Un filtrage de sécurité illusoire

Les examens de *tous* les utilisateurs étaient téléchargés, puis filtrés dans le navigateur. Le cloisonnement était donc purement cosmétique : n'importe quel compte authentifié pouvait lire les données des autres en observant les réponses de l'API.

3.2.4 Une erreur de précedence

La fonction de filtrage contenait un défaut discret :

Listing 3.2 – Précedence incorrecte entre ?? et l'opérateur ternaire

```
const eleves =  
  a?.eleves?.data ??  
  a?.eleve?.data ? [a?.eleve?.data] : []
```

La condition évaluée est `(a?.eleves?.data ?? a?.eleve?.data)`, mais la valeur retournée est toujours `[a?.eleve?.data]`. Lorsque la relation s'appelle `eleves`, le résultat est `[undefined]` et le filtrage échoue en silence.

3.2.5 Un algorithme de planification rudimentaire

Le cœur fonctionnel de l'application tenait en dix-huit lignes :

Listing 3.3 – Génération du planning, version 1

```
function generatePlan(ex) {  
  if (!ex.date || !ex.poids) return []  
  const end = new Date(ex.date)  
  const nb = Math.max(1, Math.ceil(ex.poids / 10))  
  const spanDays = Math.max(3, Math.min(14, nb * 2))  
  const sessions = []  
  for (let i = nb; i >= 1; i--) {  
    const dayOffset = Math.floor((spanDays / (nb + 1)) * i)  
    const start = new Date(end)  
    start.setDate(end.getDate() - dayOffset)  
    start.setHours(18, 0, 0, 0) // toujours 18 h, toujours 60 minutes  
    const fin = new Date(start)  
    fin.setMinutes(start.getMinutes() + 60)  
    sessions.push({ start, fin })  
  }  
  return sessions  
}
```

Cette fonction ignore la date du jour — elle peut donc produire des séances **dans le passé** —, la difficulté de la matière, les collisions entre examens et les disponibilités réelles de l'étudiant.

3.2.6 Autres points

- le type `any` était omniprésent, ce qui annulait l'intérêt de TypeScript ;
- les retours à l'utilisateur passaient par `alert()`, bloquant et non testable ;
- le dossier `node_modules` était versionné, soit 30 Mo inutiles ;
- la couverture de test se limitait à un test de fumée.

Chapitre 4

La refonte

4.1 Principes retenus

1. **Le projet doit se lancer en deux commandes.** Un projet qu'un tiers ne peut pas exécuter n'existe pas.
2. **La logique métier doit être isolée.** Le calcul du planning ne dépend ni de React ni de Strapi : il est donc testable directement.
3. **La configuration est du code.** Schémas et permissions sont versionnés, donc reproductibles.
4. **La sécurité appartient au serveur.** Aucun filtrage de données personnelles n'est délégué au navigateur.
5. **Les échecs doivent être explicites.** Une donnée invalide provoque une erreur nommée, jamais un résultat silencieusement faux.

4.2 Architecture

Listing 4.1 – Organisation du dépôt

```
revision-planner/  
|-- packages/core/ Moteur de planification -- TypeScript pur, 32 tests  
|-- apps/api/ Backend Strapi 5 -- schemas, permissions, donnees de demo  
|-- apps/web/ Frontend React 18 + Vite + Redux Toolkit, 13 tests  
'-- scripts/ Capture automatisee de la demonstration
```

Le choix d'un *monorepo* permet au front et au moteur de partager exactement les mêmes types, sans duplication ni publication de paquet intermédiaire.

4.3 Modèle de données

Type	Rôle
student	Profil étudiant, rattaché à un compte utilisateur, porteur des disponibilités et des préférences de durée
subject	Matière et sa difficulté (<i>facile, moyen, difficile</i>)
exam	Épreuve : intitulé, date, coefficient, type, matière, élève
revision-session	Créneau de révision généré, rattaché à un examen

TABLE 4.1 – Types de contenu

Les noms sont désormais fixés et versionnés dans `apps/api/src/api/*/content-types/`. Il n'y a plus rien à deviner à l'exécution.

Chapitre 5

Le moteur de planification

C'est la pièce centrale du projet. Elle est implémentée dans `packages/core` sous forme de fonctions pures, sans dépendance externe.

5.1 Volume de révision

Le temps accordé à un examen dépend de son coefficient et de la difficulté de sa matière :

$$M = \left\lceil \frac{c \times 6 \times f_d}{s} \right\rceil \times s \quad (5.1)$$

où c est le coefficient, f_d le facteur de difficulté (0,75 *facile*, 1 *moyen*, 1,35 *difficile*), s la durée d'une séance en minutes, et M le volume total accordé. L'arrondi au multiple supérieur de s traduit un choix explicite : on ne planifie pas une fraction de séance.

Examen	Coefficient	Difficulté	Séances
Anglais technique	10	facile	1
Modélisation SQL	20	moyen	2
Protocoles réseau	30	moyen	3
Mécanique quantique	35	difficile	5
Analyse numérique	40	difficile	6

TABLE 5.1 – Répartition obtenue sur le jeu de données de démonstration

5.2 Répétition espacée

Les séances sont placées à des intervalles croissants avant l'épreuve. Les écarts successifs valent 1, 2, 3, 4, ... jours, ce qui donne les décalages cumulés :

$$d_k = \sum_{i=1}^k i = \frac{k(k+1)}{2} \quad \text{soit } 1, 3, 6, 10, 15, 21, \dots \quad (5.2)$$

La révision est ainsi dense à l'approche de l'examen et de plus en plus espacée en remontant dans le temps, conformément au principe de la répétition espacée, qui favorise la mémorisation durable.

5.2.1 Adaptation à la fenêtre disponible

Pour dix séances, le décalage le plus lointain vaut $d_{10} = 55$ jours. Si l'examen a lieu dans vingt jours, viser ces dates condamne la moitié des séances. Les décalages sont donc comprimés proportionnellement :

$$d'_k = \text{round} \left(d_k \times \frac{L}{d_n} \right) \quad \text{lorsque } d_n > L \quad (5.3)$$

où L est le nombre de jours réellement disponibles. L'ordre et la progression des écarts sont conservés.

Ce point est le résultat direct d'une relecture critique : la première implémentation du moteur perdait trois séances sur dix dans ce cas de figure et les imputait, à tort, à un manque de créneaux.

5.3 Placement des séances

Pour chaque séance, l'algorithme procède ainsi :

1. calculer la date idéale, soit la date de l'examen moins le décalage ;
2. explorer les journées autour de cette date, en privilégiant d'abord la date exacte puis les jours antérieurs ;
3. pour chaque journée, parcourir les créneaux de disponibilité et y chercher un intervalle libre, en respectant les pauses et le plafond journalier ;
4. en dernier recours, balayer toute la fenêtre restante plutôt que d'abandonner la séance.

Les examens sont traités du plus proche au plus lointain : en cas de pénurie de créneaux, ce sont les échéances lointaines qui cèdent.

5.4 Garanties

Le moteur garantit, et ses tests le vérifient :

- aucune séance dans le passé, ni après la date de l'examen concerné ;
- une marge de repos configurable avant chaque épreuve ;
- aucun chevauchement entre deux séances, y compris pour des examens différents ;
- le respect des créneaux, des pauses et du plafond journalier ;
- un résultat déterministe à données identiques.

5.5 Diagnostics

Lorsqu'une séance ne peut pas être placée, le moteur ne l'omet pas en silence : il renvoie une cause explicite — *examen déjà passé*, *aucune disponibilité*, ou *temps insuffisant* — affichée à l'utilisateur avant l'enregistrement.

5.6 Validation des entrées

Un coefficient non fini provoque une erreur nommée plutôt qu'un résultat aberrant :

```
if (!Number.isFinite(exam.weight)) {
  throw new RangeError(
    'Coefficient invalide pour "${exam.name}" : ${exam.weight}'
  )
}
```

Cette vérification est née d'une revue : sans elle, un coefficient NaN faisait disparaître l'examen du planning sans le moindre avertissement — le défaut même reproché à la version 1.

Chapitre 6

Le backend

6.1 Permissions appliquées par le code

Les permissions du rôle *authenticated* sont posées au démarrage, et non cochées à la main dans l'administration :

Listing 6.1 – `src/bootstrap/permissions.ts`, extrait

```
const AUTHENTICATED_ACTIONS = [
  'api::exam.exam.find', 'api::exam.exam.create',
  'api::exam.exam.update', 'api::exam.exam.delete',
  'api::revision-session.revision-session.find',
  /* ... */
]
```

L'opération est idempotente. Sur une base vierge, le démarrage applique quinze permissions authentifiées et une publique, puis injecte les données de démonstration. C'est ce qui rend l'installation reproductible et supprime les erreurs 403 de la version 1.

6.2 Cloisonnement des données

Chaque requête est filtrée côté serveur par le profil étudiant du compte appelant :

Listing 6.2 – Contrôleur `exam`, méthode `find`

```
async find(ctx) {
  const student = await requireStudent(strapi, ctx)
  ctx.query = {
    ...ctx.query,
    filters: {
      ...ctx.query?.['filters'],
      student: { documentId: { $eq: student.documentId } },
    },
  }
  return super.find(ctx)
}
```

En écriture, la relation `student` est imposée par le serveur : un examen ne peut pas être créé pour autrui, ni réaffecté. Une requête sans jeton reçoit un 403.

6.3 Données de démonstration

Le jeu de démonstration emploie des dates *relatives* au jour du démarrage. La démonstration reste donc pertinente quelle que soit la date à laquelle le dépôt est cloné, au lieu d'afficher des examens périmés.

Chapitre 7

L'interface

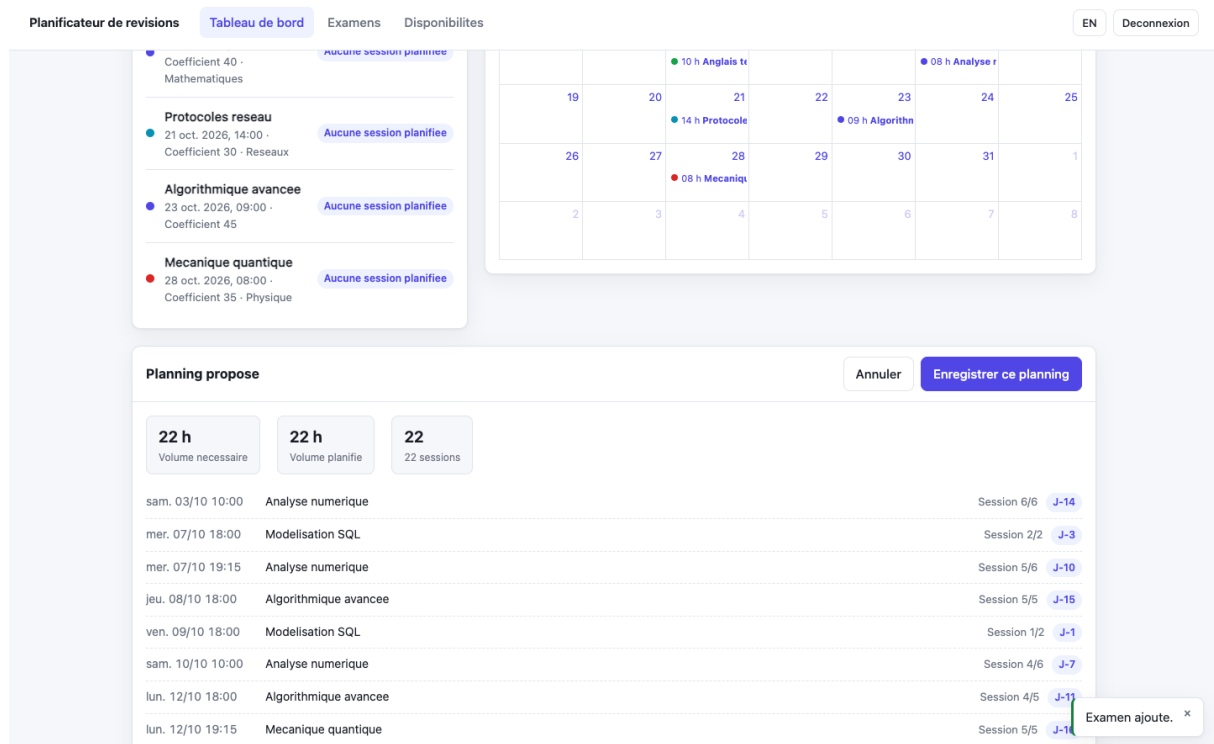


FIGURE 7.1 – Prévisualisation du planning avant enregistrement : volume nécessaire, volume planifié, et chaque séance datée avec son rang et son écart à l'examen.

7.1 Prévisualisation avant enregistrement

La version 1 écrivait directement en base puis affichait `alert('Plan généré')`. L'étudiant ne voyait ni ce qui avait été créé, ni les échecs partiels. La refonte introduit une étape de prévisualisation : le planning est calculé, présenté avec ses diagnostics, et n'est enregistré qu'après confirmation.

7.2 Déclaration des disponibilités

Cette page n'existait pas auparavant : les séances étaient systématiquement placées à 18 h pour une heure, sans jamais demander son avis à l'étudiant.

Planificateur de révisions Tableau de bord Examens Disponibilités EN Deconnexion

Mes disponibilités

Indiquez quand vous acceptez de réviser. Le planning ne proposera que ces créneaux.

Durée d'une session (minutes)	Maximum par jour (minutes)
60	180

Jour	Debut	Fin	
Lundi	06:00 PM	09:00 PM	Retirer
Mardi	06:00 PM	09:00 PM	Retirer
Mercredi	06:00 PM	09:00 PM	Retirer
Jeudi	06:00 PM	09:00 PM	Retirer
Vendredi	06:00 PM	09:00 PM	Retirer
Dimanche	10:00 AM	01:00 PM	Retirer
Samedi	10:00 AM	01:00 PM	Retirer

Ajouter un créneau Enregistrer

Examen ajouté. ✕

FIGURE 7.2 – Édition des créneaux hebdomadaires. Le planning ne propose que ces plages.

7.3 Notifications

Les `alert()` ont été remplacés par une pile de notifications non bloquantes, gérée dans le *store* Redux. Outre le confort, ce choix rend l'interface vérifiable automatiquement : un `alert()` suspend le navigateur et interrompt tout pilotage par Playwright.

Chapitre 8

Qualité

8.1 Tests

Paquet	Tests	Portée
packages/core	32	volume, espacement, garanties, cas limites, validation des entrées
apps/web	13	sérialiseur de requêtes, parité des traductions, conversion des données
Total	45	

TABLE 8.1 – Couverture de test

Un enseignement de la relecture mérite d’être noté : les premiers tests écrits validaient surtout mes propres hypothèses. Ce sont les tests portant sur les *cas limites* — coefficient invalide, fenêtre trop courte, calendrier saturé — qui ont révélé les véritables défauts.

8.2 Démonstration reproductible

Le script `scripts/capture-demo.mjs` pilote l’application avec Playwright, enregistre une image à chaque étape, puis assemble une vidéo et un GIF avec ffmpeg. Il réinitialise les données avant de filmer, ce qui le rend rejouable à l’identique. La démonstration se régénère donc après chaque évolution de l’interface, au lieu d’être un enregistrement manuel vite périmé.

Chapitre 9

Difficultés rencontrées

9.1 Pendant le stage

La difficulté principale fut le volume de notions à acquérir simultanément : typage statique, programmation par composants, gestion d'état centralisée, protocole REST, relations entre types de contenu, authentification par jeton. Chaque obstacle technique se doublait d'un obstacle conceptuel.

Le dialogue entre le front et Strapi fut le point le plus coûteux, en particulier la structure des réponses, les relations à peupler explicitement, et la configuration des permissions — dont la méconnaissance explique les contournements relevés au chapitre 3.

9.2 Pendant la refonte

Un cache de paquets corrompu

Une installation interrompue avait laissé des paquets incomplets. Trois pannes sans rapport apparent en découlaient : `ts.sys undefined`, une erreur de schéma `yup`, et un build Strapi impossible. Une réinstallation propre les a toutes résolues d'un coup. Leçon : remonter à la cause commune plutôt que traiter les symptômes.

Une variable d'environnement vide

`DATABASE_FILENAME=` renvoyait une chaîne vide plutôt que la valeur par défaut ; Strapi tentait alors d'ouvrir un *répertoire* comme base de données.

Un dossier synchronisé

Le projet résidait initialement sur un Bureau synchronisé par iCloud Drive. Les démons de synchronisation génèrent des événements système que les observateurs de fichiers de Vite et de Strapi interprètent comme des modifications, provoquant des redémarrages en boucle. Un projet Node n'a pas sa place dans un dossier synchronisé.

Chapitre 10

Bilan et perspectives

10.1 Ce que le stage a apporté

En un mois chez Anypli, je suis passé d’une absence totale d’expérience en développement web à une application complète et fonctionnelle : interface React typée, backend Strapi, authentification, internationalisation, affichage calendaire. Cet apprentissage accéléré constitue le véritable acquis du stage, et la première version doit être jugée à cette aune.

10.2 Ce que la refonte a apporté

La refonte relève d’une autre compétence : non plus faire fonctionner, mais rendre *fiable*. Isoler la logique métier pour la tester, versionner la configuration pour la rendre reproductible, placer la sécurité au bon endroit, rendre les échecs explicites — autant de réflexes qui ne s’acquièrent qu’en reprenant son propre code avec du recul.

	Version 1	Version 2
Backend	à configurer à la main	versionné dans le dépôt
Lancement	impossible sans réglages	<code>npm install && npm run dev</code>
Planification	18 lignes	moteur isolé et testé
Séances passées	générées	impossibles par construction
Filtrage	navigateur	serveur
Tests	1	45
TypeScript	<code>any</code> omniprésent	strict, sans <code>any</code>

TABLE 10.1 – Comparaison des deux versions

10.3 Perspectives

- ajuster les constantes du moteur — six minutes par point de coefficient et les facteurs de difficulté sont des choix de conception, non des valeurs établies ; les rendre configurables par l’étudiant serait un progrès ;
- gérer explicitement les fuseaux horaires, aujourd’hui alignés sur celui du navigateur ;
- tenir compte de l’avancement réel des séances pour replanifier ce qui n’a pas été fait ;
- exporter le planning au format iCalendar.

Code source : <https://github.com/aziz3r/revision-planner>